

Respaldo formal de la teoría de categorías

Auditoría completa: qué afirma Python y qué demuestra Lean

Leonardo Jiménez Martínez
en colaboración con Claude Opus 5 (Anthropic)

23 de agosto de 2026

Resumen

METAMATEMÁTICO implementa teoría de categorías a mano: no hay ninguna librería detrás de `nucleo/graph/` ni de `nucleo/mes/`, solo `dataclasses`, diccionarios y recorridos en anchura. Eso hace que cada propiedad categórica sea una afirmación del autor hasta que Lean 4 la respalda. Este documento audita, operación por operación, cuáles lo están. El resultado es 38 de 39 (97 %). Todas las cifras proceden de `data/respaldo_lean.json`, generado por `scripts/auditar_respaldo_lean.py`; ninguna se escribe a mano.

1. Por qué esta auditoría existe

Tres defectos reales, encontrados en código que funcionaba y no fallaba ruidosamente:

- `reachable_from` seguía morfismos de tipo `TRANSLATION`, lo que producía colímites espurios como `= tactic-simp`.
- `build_join_for_pattern` *fabricaba* vértices cuando no encontraba el colímite, y por eso la iteración no convergía.
- `patterns.py` exigía $\exists h$ donde la propiedad universal pide $\exists! h$.

Ninguno lo habría cometido una librería madura, y ninguno se detectó leyendo el código: los tres salieron al medir y al formalizar.

2. Resumen

	Operaciones	%
Respaldadas por un teorema	38	97
Sin respaldo formal	1	3
Mapeo roto	0	0
Total auditado	39	100

Declaraciones formales disponibles en `MetamathProver/CategoryFoundations/`: **197**.

3. Operaciones respaldadas

Módulo	Operación en Python	Teorema en Lean
	<i>category.py</i>	

Módulo	Operación en Python	Teorema en Lean
	<code>identity</code>	<code>preorder_is_thin</code> id_a existe y es unico en Hom(a,a)
	<code>compose</code>	<code>preorder_comp_trans</code> la composicion existe y es asociativa por transitividad
	<code>hom</code>	<code>thin_unique_hom</code> Hom(a,b) tiene a lo sumo un elemento
	<code>reachable_from</code>	<code>reachable_refl</code> el cierre transitivo-reflexivo es el preorden
	<code>is_preorder_leq</code>	<code>preorder_le_of_hom</code> $a \leq b$ syss hay morfismo $a \rightarrow b$
	<code>classify_link</code>	<code>simple_of_cluster</code> simple = INDUCIDO por un cluster entre descomposiciones (Ehresmann); la lectura por factorizacion esta en <code>composite_of_simple_is_simple</code>
	<code>verify_axioms</code>	<code>path_category_axioms</code> los axiomas de categoria valen en la categoria libre del quiver
	<code>apply_complexity_order</code>	<code>hierarchy_well_founded</code> la iteracion de cn alcanza punto fijo
	<code>verify_pillar_multiplicity</code>	<code>MultiplicityPrinciple</code> existen patrones homologos con el mismo colimite
<i>complexity.py</i>		
	<code>find_cocones</code>	<code>isCocone</code> co-cono = cota superior de las componentes
	<code>find_colimit</code>	<code>isColimitInFiniteCategory</code> colimite = co-cono minimal, decidible en categoria finita
	<code>find_existing_join</code>	<code>join_implies_colimit</code> el join es el colimite
	<code>build_join_for_pattern</code>	<code>colimit_is_least_upper_bound</code> la universalidad es un TEST, no una construccion
	<code>build_hierarchy_to_fixpoint</code>	<code>hierarchy_well_founded</code> la iteracion termina
	<code>compute_complexity_order</code>	<code>cn_join_gt_component</code> $cn(\text{join}) > cn$ de cada componente
	<code>_find_upper_bounds</code>	<code>colimit_is_upper_bound</code> las cotas superiores del patron
<i>patterns.py</i>		
	<code>verify_cocone</code>	<code>isCocone</code> el co-cono conmuta
	<code>is_join</code>	<code>join_is_minimal_upper_bound</code> minimalidad entre las cotas superiores
	<code>verify_universal_property</code>	<code>join_mediating_morphism_unique</code> el mediador existe y es UNICO (!h, no h)
	<code>build_colimit</code>	<code>isJoin_iff_nonempty_isColimit</code> IsJoin equivale a IsColimit de Mathlib
	<code>verify_multiplicity_principle</code>	<code>multiplicidad_necesaria_para_complejidad</code> sin Principio de Multiplicidad todo compuesto de simples es simple: MP es condicion NECESARIA de la complejidad, no un adorno sobre robustez

Módulo	Operación en Python	Teorema en Lean
	<code>detect_pattern_in_graph</code>	<code>enlaces_complejos_existen</code> modelo concreto: dos enlaces simples cuya composición no es simple
	<code>_connected_by_cluster</code>	<code>complex_needs_unconnected</code> un enlace complejo exige descomposiciones intermedias NO conectadas
<i>complexity.py</i>	<code>_detect_convergence_patterns</code>	<code>MP_alcanzable_en_preorden</code> emite también las descomposiciones por subconjunto: MP es alcanzable
	<code>compute_complexity_order</code> (multi)	<code>hierarchy_well_founded_multi</code> con varias descomposiciones, el MAXIMO converge; asignar oscilaba
<i>patterns.py</i>	<code>son_homologos</code>	<code>Homologos</code> homologia de Ehresmann: mismo colimite
	<code>find_homologous_patterns</code>	<code>colimite_unico</code> el colimite de un patron es unico, luego la homologia esta bien definida
<i>functor.py</i>	<code>construir_functor</code>	<code>proyeccion_es_functor</code> una proyección monótona es funtor en categorías thin
	<code>verificar_functorialidad</code>	<code>proyeccion_es_functor</code> las dos leyes de funtor
	<code>verificar_preservacion_colimites</code>	<code>functor_preserves_cocone</code> los funtores preservan co-conos
	<code>verificar_preservacion_colimites</code> (minimalidad)	<code>functor_not_preserves_join</code> y NO preservan la minimalidad: contraejemplo finito
	<code>CategoriaAgentes.alcanzables_desde</code>	<code>refleja_no_alcanzabilidad</code> pi refleja la no-alcanzabilidad
<i>evolution.py</i>	<code>TransitionFunctor</code>	<code>comp</code> functor parcial entre instantáneas: none = objeto eliminado
	<code>compose</code>	<code>comp_assoc</code> la composición de transiciones es asociativa
	<code>verify_compatibility</code>	<code>compatible_iff</code> compatibilidad (Def 3.2) = ley de funtor sobre el orden del tiempo
	<code>apply_option</code>	<code>eliminado_es_absorbente</code> lo eliminado no vuelve: ninguna composición posterior lo recupera
	<code>measure_emergence</code>	<code>soloCrece_cadena</code> emergence_ratio es comparable entre tiempos si nada se elimina
	<code>detect_complex_links</code>	<code>complex_needs_unconnected</code>

Módulo	Operación en Python	Teorema en Lean
		un enlace complejo exige descomposiciones intermedias no conectadas

4. Sin respaldo formal

Lo que sigue no está demostrado. Está aquí porque un hueco anotado es distinguible de un hueco olvidado, y `tests/test_respaldo_lean.py` falla si esta lista cambia sin que alguien lo declare.

Módulo	Operación	Qué afirma
<code>patterns.py</code>	<code>campos_operativos_isomorfismos</code>	homotopía estructural, NO la homología de Ehresmann

Las tres pertenecen a `evolution.py`, es decir, al núcleo evolutivo de Ehresmann: complexificación, funtor de transición y detección de emergencia. Es el frente abierto.

5. Un resultado negativo que conviene leer

Al formalizar la distinción simple/complejo de Ehresmann se intentó demostrar el teorema central de la emergencia: que la composición de dos enlaces simples puede no ser simple. **Lean lo refutó.**

La razón es estructural, no un descuido del diseño. Si $a \leq k \leq b$ y $b \leq n \leq d$, la transitividad da $k \leq d$, de modo que el propio k factoriza $a \rightarrow d$. El teorema verdadero es el contrario:

$$\text{composite_of_simple_is_simple} : \text{simple}(a, b) \wedge \text{simple}(b, d) \Rightarrow \text{simple}(a, d).$$

En una categoría delgada los enlaces simples son cerrados por composición, y la distinción de Ehresmann pierde su contenido dinámico: los complejos no *emergen* al componer, solo aparecen donde *faltan* clústeres.

La delgadez es la hipótesis que hace que `join` = colímite, y de ella dependen `JoinColimit.lean` e `IsColimitBridge.lean`. Recuperar la emergencia exigiría abandonarla admitir más de un morfismo entre dos habilidades y distinguir *cómo* se llega, no solo si se llega, lo que es un cambio de modelo y no un ajuste. Queda registrado como límite conocido.

6. Declaraciones con `sorry`

- Ninguno.

Se cuentan preguntándole al compilador, no con `grep`: un `sorry` dentro de un comentario no es un `sorry`.

Reproducibilidad

- `scripts/auditar_respaldo_lean.py` la auditoría; genera el JSON.
- `scripts/generar_reporte_respaldo.py` genera este documento.
- `tests/test_respaldo_lean.py` falla si el mapeo se rompe o si el respaldo retrocede.
- `MetamathProver/CategoryFoundations/` 197 declaraciones.